ROUND 2

72 Hours Code Challenge

Associate AI Forward Deployed Engineers · AI Forward Deployed Engineer

Problem statement

Read it carefully. Your build should cover every point below.

Business Context / Problem
Monitor seller service levels, order fulfillment, disputes, commissions and payout eligibility.

Objective and Scope
Build a working solution for this end-to-end workflow: Monitor seller service levels, order fulfillment, disputes, commissions and payout eligibility. The solution should expose key decisions and exceptions, preserve traceability, and produce measurable outputs. Medium complexity means multiple related entities, conditional rules, at least one exception path and meaningful evaluation metrics.

Primary Actors / Systems
Customer, store/warehouse operator, merchandising/e-commerce analyst and operations manager

Business Workflow and Rules
1. Generate/reference master data first: Seller, ProductListing, Order, OrderLine, Fulfillment.
2. Generate dependent transaction/event records using valid parent IDs.
3. Apply business logic: Create sellers and listings; generate orders across sellers; split multi-seller orders into fulfillment records; record shipment/delivery and reviews; define commission and reserve rules; create disputes/chargebacks; calculate payout lines after delivery and hold windows.
4. Persist state changes as explicit records/events where relevant; do not overwrite history.
5. Generate outcomes and ground-truth labels from documented rules, not independent random assignment.

Entities and Relationships
Core entities: Seller, ProductListing, Order, OrderLine, Fulfillment, CustomerReview, Dispute, CommissionRule, Payout, PayoutLine. Define a primary key for every entity and foreign keys for every relationship. Document cardinality (1:1, 1:N, N:M), optional relationships, effective dates and lifecycle states. Use bridge tables for many-to-many relationships; retain event history for state changes; validate all joins. For deliberate anomaly cases, keep an issue_type/ground_truth label rather than silently corrupting unrelated rows.

ID / Nomenclature Conventions
SELL-000001; LIST-<seller-id>-000001; ORD-2026-000001; FUL-<order-line-id>; DSP-2026-000001; PAYOUT-2026-000001.

Synthetic Data Generation Instructions (Granular)
Step 1 — Write a data dictionary for: Seller, ProductListing, Order, OrderLine, Fulfillment, CustomerReview, Dispute, CommissionRule, Payout, PayoutLine. For each field specify type, required/optional, allowed values, units, uniqueness and validation rule.
Step 2 — Create parent/master tables first, then child and event tables. Apply this domain-specific generation logic: Create sellers and listings; generate orders across sellers; split multi-seller orders into fulfillment records; record shipment/delivery and reviews; define commission and reserve rules; create disputes/chargebacks; calculate payout lines after delivery and hold windows.
Step 3 — Apply deterministic IDs and naming: SELL-000001; LIST-<seller-id>-000001; ORD-2026-000001; FUL-<order-line-id>; DSP-2026-000001; PAYOUT-2026-000001. Keep counters zero-padded, IDs immutable, and referenced IDs exactly consistent across files.
Step 4 — Generate the baseline volume: Generate 2,000 sellers, 20,000 listings, 100,000 orders and 10,000 disputes. Include split orders, late shipment, returns, reserve holds and seller suspension. Use a fixed seed (e.g. 20261022) and parameterize scale so starter and full datasets use the same generator.
Step 5 — Generate valid baseline records first, then inject labelled edge cases: Payout before delivery/hold release; commission applied twice; suspended seller continues listing; dispute not deducted; payout total does not equal lines.
Step 6 — Calculate derived values (totals, balances, scores, statuses and dates) from source records rather than generating them independently.
Step 7 — Produce a separate ground_truth.csv/json for known anomalies, expected decisions or labels; never leak it into input features.
Step 8 — Run validation: primary-key uniqueness, foreign-key integrity, allowed enums, date ordering, arithmetic reconciliation, lifecycle transitions and duplicate checks.
Step 9 — Export CSV/JSON/SQL seed data plus README, schema/data dictionary, generator command, fixed seed, source manifest and validation summary.
Step 10 — Regenerate twice with the same seed and compare row counts/checksums to

Deadline

01	04	20
DAYS	HOURS	MINS

Thu 29 Oct 2026, 08:22 (UTC)

Open

How it works

1. Build the solution locally.
2. Push it to a public GitHub repository.
3. Submit the link here before the deadline.

Questions?

Stuck on the brief? Our recruitment team can help.

contactus@yottaflex.ai

Back to your application →

Help & FAQs →

demonstrate reproducibility.

Suggested Volume and Coverage
Generate 2,000 sellers, 20,000 listings, 100,000 orders and 10,000 disputes. Include split orders, late shipment, returns, reserve holds and seller suspension.

Required Edge Cases / Negative Scenarios
Payout before delivery/hold release; commission applied twice; suspended seller continues listing; dispute not deducted; payout total does not equal lines.

Validation Rules / Acceptance Criteria
Seller metrics are reproducible; payouts reconcile to order value, fees, refunds and holds; seller status rules are enforced.

Evaluation Focus for Generated Data
Review: (a) realistic distributions and meaningful variation; (b) consistent IDs and valid joins; (c) dependency/lifecycle consistency across entities; (d) correct derived values and reconciliation; (e) normal, boundary, failure and exception coverage; (f) separate ground truth; (g) fixed-seed reproducibility, documentation and lawful provenance.

Public Data Guidance / Lawful Use
External data is optional. If used, record source URL, dataset/version, retrieval date, license/terms, attribution, transformations and fields discarded. Use only data you are legally permitted to access and redistribute; do not bypass access controls or collect confidential/personal information. If licensing or suitability is unclear, use synthetic data and cite public reference data only for schema inspiration.

BEFORE YOU SUBMIT, MAKE SURE YOU HAVE

- Pushed your code to a **public** GitHub repository
- Added a README explaining how to run it
- Included any tests the problem asks for

Candidate FAQ (PDF)

Submission Guidelines (PDF)

Submit your work

You can submit once, so check everything before you do.

GitHub repository URL *

https://github.com/username/repo

The repository must be **public** so we can review it.

Your submission is final once sent.

Submit Challenge

Copyright © 2026 YottaFlex.ai Inc All Rights Reserved.

Powered by YottaFlex.ai